Architecture des Systèmes d'Exploitation : Processus, Cycle de Vie, Filiation et Signaux POSIX

Thème principal : Gestion des processus sous Unix/Linux, multitâche préemptif, filiation fork/exec et signaux POSIX

Synthèse exécutive : Ce cours de R1.04 traite du fonctionnement fondamental des systèmes d'exploitation de type Unix/Linux au niveau de la gestion des processus. Il détaille la transition d'un programme statique (fichier exécutable stocké sur disque) vers un processus dynamique résidant en mémoire vive (RAM). Pour exécuter simultanément de nombreuses applications sur un nombre restreint de cœurs processeurs, le noyau Linux s'appuie sur le multitâche préemptif et un ordonnanceur (scheduler) répartissant les quanta de temps processeur selon un cycle de vie en trois états (Élu, Prêt, Bloqué). Le cours explique également la généalogie stricte parent-enfant des processus issue de systemd (PID 1), le double mécanisme de création via fork() et exec(), ainsi que le contrôle et la communication inter-processus par les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, PCB et Multitâche

Pour gérer l'exécution concurrente de multiples programmes, le système d'exploitation convertit les fichiers exécutables du disque dur en entités dynamiques appelées processus lors de leur chargement en mémoire RAM. Afin de simuler une simultanéité parfaite sur un nombre limité de cœurs CPU, le noyau Linux applique le multitâche préemptif. L'ordonnanceur (scheduler) attribue à chaque processus prêt une très courte fraction de temps processeur (quantum de temps). Le cycle de vie d'une tâche est régi par trois états : 1. Élu (Running) : Le processus occupe le processeur et exécute son code binaire. 2. Prêt (Ready) : Le processus possède toutes ses ressources en RAM et attend la décision de l'ordonnanceur pour obtenir un quantum CPU. 3. Bloqué (Waiting/Blocked) : Le processus est mis en pause à la suite d'une demande d'opération longue (lecture disque, saisie clavier, attente réseau). Le CPU est immédiatement libéré pour une autre tâche. Lorsque l'événement attendu survient, le processus ne repasse JAMAIS directement à l'état Élu : il réintègre impérativement la file des processus Prêts pour attendre son tour d'ordonnancement.

- Programme vs Processus : Un programme est un fichier d'instructions statique et passif stocké sur disque ; un processus est l'instance vivante et active exécutée en RAM.
- Multitâche Préemptif & Ordonnanceur : Découpage du temps CPU par le noyau en minuscules quanta attribués équitablement aux processus.
- Process Control Block (PCB) : Structure interne du noyau conservant l'état processeur, la carte mémoire (pile, tas, code), la priorité, le PID, PPID, UID et les descripteurs de fichiers.
- Modèle des 3 États : Élu (Running), Prêt (Ready), Bloqué (Waiting/Blocked). Règle absolue : un processus Bloqué repasse toujours par l'état Prêt avant de revenir à l'état Élu.

Cas concret enseignant : *Analogie du livre de recettes de cuisine : Le programme stocké sur le disque dur est comparable à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en mémoire RAM sur le plan de travail, consommant du temps processeur CPU pour la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Filiation, Arborescence et Mécanismes de Création (fork et exec)

Sous Unix/Linux, la création de processus s'inscrit dans une généalogie stricte. Au démarrage du système, le noyau initialise le premier processus utilisateur nommé 'systemd' (ou 'init'), se voyant attribuer de manière immuable le PID 1. Aucun processus ne peut apparaître de façon isolée : tout nouveau processus est engendré par un processus parent. Lorsqu'une tâche est lancée depuis le terminal (shell), la création s'effectue en deux phases distinctes : 1.

Le père appelle fork() : le noyau clone à l'identique le contexte mémoire et l'environnement du père. Le fils reçoit un nouveau PID et son PPID prend la valeur du PID du père. 2. Le fils appelle exec() : le noyau écrase l'image mémoire héritée pour y charger le nouveau programme à exécuter (ex: /bin/ls). Le fils conserve son PID original mais exécute désormais un code totalement nouveau.

- Arborescence et Identifiants (PID / PPID) : Tout processus possède un Process ID unique et pointe vers son Parent Process ID. L'arbre entier remonte à la racine unique systemd/init (PID 1).
- Appel système fork() : Duplication exacte par le noyau du processus père pour engendrer un processus fils doté d'un nouveau PID.
- Appel système exec() : Remplacement de l'espace mémoire du processus appelant par le code binaire d'un nouveau programme.

Cas concret enseignant : *Exemple concret du terminal : Lors de la saisie d'une commande comme 'ls -l' dans Bash, le shell (père) exécute d'abord fork() pour se dédoubler en un processus fils identiquement configuré, puis ce fils appelle immédiatement exec('/bin/ls') pour remplacer son propre code par celui du programme de listage.*

3. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX constituent le moyen le plus simple d'envoyer une notification ou un ordre asynchrone à un processus. À la réception d'un signal, le processus peut : 1. Laisser le système appliquer l'action par défaut (généralement l'arrêt du programme). 2. Ignorer le signal. 3. Intercepter le signal en déclenchant un gestionnaire personnalisé (signal handler), ce qui permet de sauvegarder un document ou de fermer des connexions proprement. Cependant, pour éviter qu'un processus malveillant ou planté ne devienne incontrôlable, la norme POSIX réserve deux signaux incapturables : SIGKILL (Signal 9 - destruction immédiate) et SIGSTOP (Signal 19 - mise en pause). Ces deux signaux sont interceptés directement par le noyau Linux et ne sont jamais transmis au code utilisateur.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau pour avertir ou ordonner une action à un processus.
- Gestionnaire de signal (Signal Handler) : Routine enregistrée dans l'application pour capturer (catch) un signal et exécuter une fonction personnalisée au lieu du traitement par défaut.
- SIGINT (2) & SIGTERM (15) : Interruptions douces autorisant le nettoyage et la sauvegarde propre avant arrêt.
- SIGKILL (9) & SIGSTOP (19) : Signaux absolus directement traités par le noyau, STRICTEMENT IMPOSSIBLES à intercepter, ignorer ou masquer.

Cas concret enseignant : *Analogie de l'expulsion du bâtiment : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (Signal 9) représente l'intervention de la brigade de sécurité d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Dans la syntaxe du shell Linux, l'option qui spécifie le numéro du signal à envoyer doit obligatoirement être précédée d'un tiret (-). Si vous écrivez 'kill 9 1234', la commande 'kill' interprète '9' et '1234' comme deux PIDs distincts auxquels elle tente d'envoyer le signal par défaut (SIGTERM 15). N'existant pas ou n'appartenant pas à vos droits, le PID 9 provoque une erreur de permission ou d'inexistence. En écrivant '-9', vous indiquez au shell que 9 est le numéro de signal (SIGKILL) à adresser au processus 1234.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' dans le système et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus zombie est une tâche dont l'exécution est terminée, mais dont le père n'a pas encore lu le code de sortie via l'appel système wait(). Le zombie n'utilise plus de mémoire RAM ni de temps CPU, mais son PID reste inscrit dans la table des processus du noyau. Il est impossible de tuer un zombie avec la commande kill (même avec kill -9) car le processus est DEJA mort ! Pour supprimer un zombie, il faut forcer son processus père à lire son état de fin, ou tuer le père : le zombie sera alors adopté par systemd (PID 1) qui supprimera immédiatement son entrée de la table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&' à la fin de la première ligne, détaillez le comportement produit par l'envoi du signal -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Démarche de résolution :
Étape 1 : Le symbole '&' en fin de ligne ordonne au shell d'exécuter la commande en arrière-plan (background job). Le shell crée le processus via fork/exec, affiche son numéro de job et son PID (ex: 4521), puis redonne immédiatement la main à l'utilisateur sans bloquer le terminal. Étape 2 : La commande 'ps -ef | grep sleep' interroge la table des processus du noyau et filtre les lignes contenant la chaîne 'sleep'. Elle permet de vérifier que le processus tourne bien, en identifiant son PID (4521), son PPID (PID du shell Bash) et son état actuel (Prêt ou Bloqué/Élu). Étape 3 : La commande 'kill -15 4521' envoie le signal POSIX n°15 (SIGTERM) au processus 4521. N'ayant pas de gestionnaire d'interception sur-mesure défini par le développeur de la commande 'sleep', le processus applique le comportement système par défaut : la fermeture propre et la restitution de toutes ses ressources.

```
#!/bin/bash
# Séquence de démonstration de gestion de tâches en arrière-plan et envoi de signaux

# 1. Lancement de la tâche en arrière-plan
sleep 600 &

# 2. Identification du PID du processus
ps -ef | grep sleep

# 3. Émission du signal SIGTERM (Signal 15) d'arrêt propre
kill -15 4521
```

Résultat : Le processus 'sleep 600' (PID 4521) est définitivement arrêté et retiré de la RAM. Une exécution ultérieure de 'ps -ef | grep sleep' confirme qu'il a complètement disparu de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement du système ?

R : Le tout premier processus utilisateur est nommé systemd (ou init sur d'anciennes distributions) et porte obligatoirement le PID 1.

Q : Quel signal POSIX est généré et transmis au processus en premier plan lorsque l'utilisateur effectue la combinaison de touches 'Ctrl + C' dans son terminal ?

R : La combinaison 'Ctrl + C' génère et transmet le signal SIGINT (Signal n°2) pour demander une interruption douce du processus.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : SIGKILL est traité directement au niveau le plus bas du noyau (kernel) sans être transmis au processus applicatif. Aucun code utilisateur ne peut donc l'intercepter, l'ignorer ou le détourner.